推测性锁省略：实现高度并发的多线程执行

出版商： IEEE

引用此内容 PDF

R·拉杰瓦尔； JR·古德曼 所有作者

151 引用文件 118 引用专利 1810 满的文本浏览量

抽象的

文档章节

1 介绍

2 背景

3 启用并发临界区

4 为什么 SLE 能正常工作?

5 实施SLE

显示完整大纲

作者

数据

参考

引用

关键词

指标

更多类似内容

抽象的：
由于临界区导致的线程串行化是多线程程序实现高性能的根本瓶颈。动态地，这种串行化可能是不必要的，因为这些临界区本可以在不使用锁的情况下安全地并发执行。当前的处理器无法充分利用这种并行性，因为它们缺乏动态检测此类错误线程间依赖关系的机制。我们提出了一种名为推测性锁省略（SLE）的新型微架构技术，旨在消除动态的、不必要的锁引起的串行化，从而实现高度并发的多线程执行。其核心思想是，并非总是需要获取锁才能正确执行。同步指令被预测为不必要的并被省略。这使得多个线程可以并发执行受同一锁保护的临界区。由于线程间数据冲突导致的推测错误通过现有的缓存机制进行检测，并使用回滚机制进行恢复。成功的推测性省略无需获取锁即可验证并提交。SLE 可以完全在微架构中实现，无需指令集支持和系统级修改，对程序员透明，并且只需要极少的额外硬件支持。 SLE 可以为程序员提供一条快速编写正确、高性能多线程程序的途径。

发表于： 第34届ACM/IEEE微体系结构国际研讨会论文集。MICRO-34

会议日期： 2001年12月1日至5日

DOI： 10.1109/MICRO.2001.991127

添加到 IEEE *Xplore* 的日期： 2002 年 8 月 6 日

出版商： IEEE

印刷版ISBN： 0-7965-1369-7

会议地点： 美国德克萨斯州奥斯汀市

印刷版 ISSN： 1072-4451

第一部分
介绍

对多线程软件的显式硬件支持，无论是共享内存多处理器还是硬件多线程架构，都变得越来越普遍[9]、[21]、[38]、[2]。随着此类支持的普及，应用程序开发人员需要利用这些发展成果，采用多线程编程。虽然服务器工作负载历来展现出丰富的线程级并行性，但越来越多的证据表明，如果程序员致力于利用这些新兴架构，桌面应用程序也可以展现出这种并行性。

在多线程程序中，同步机制（通常是锁）常用于确保线程在代码的关键部分拥有对共享数据的独占访问权限。一个线程获取锁，执行其关键部分，然后释放锁。所有其他线程都会等待锁，直到第一个线程完成其关键部分，从而实现访问的串行化，使得整个关键部分看起来像是原子执行的。

由于多种原因，多个线程在临界区内并发访问共享数据结构实际上可能不会发生冲突，并且此类访问不需要序列化。图 1展示了两个这样的例子。 图 1a展示了来自多线程应用程序 ocean [40]的一个示例。由于存在对共享对象的存储指令（第 3 行），因此需要加锁。然而，大多数动态执行不会执行存储操作，因此不需要加锁。此外，多个线程可以在持有共享对象锁的情况下更新共享对象的不同字段，并且这些更新通常不会发生冲突。 图 1b展示了这样一个涉及哈希表更新的示例。该示例类似于 SHORE（一个数据库对象存储库 [5]）中的线程哈希表实现。

在这些例子中，乱序处理器中的传统推测执行无法利用并行性，因为线程会先等待空闲锁，然后以串行方式获取锁。目前尚无机制可以检测这种并行性。频繁的串行化会降低即使是经过优化的多线程应用程序的性能[37][20]，并且在保守同步机制存在的情况下，性能下降会更加严重。

PDF 帮助

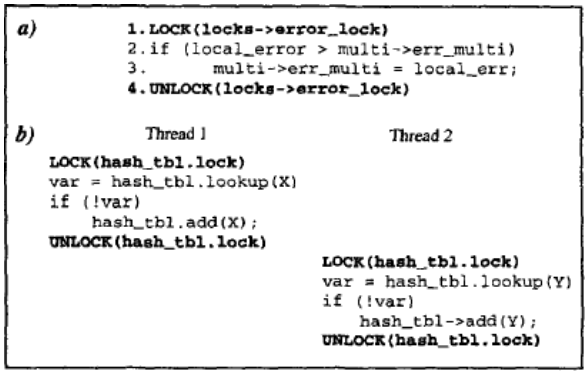

图1。两个潜在的并行性被动态不必要的同步所掩盖的例子。

在开发多线程程序时，程序员必须在性能和代码开发时间之间做出权衡。虽然多线程编程可以提高吞吐量，但需要一定的专业知识来确保程序线程之间的正确交互。这种专业知识要求通常高于大多数单线程程序，因为线程间的数据结构共享往往微妙而复杂。程序员可以通过使用保守的技术[17]来避免许多复杂性，同时确保程序的正确性。这样做可以更快、更容易地编写出正确运行的程序，但由于对线程执行引入了不必要的、由同步引起的串行化约束，限制了程序中的线程级并行性：在动态执行过程中，线程之间可能不存在数据冲突。

理想情况下，程序员可以使用频繁且保守的同步机制来编写显然正确的多线程程序，而工具则会自动移除所有此类保守的同步机制。因此，即使程序员使用简单的方案来编写正确的代码，同步也只会在确保正确性所必需的情况下执行；并且不会因为动态地进行不必要的同步而降低性能。

本文展示了如何利用硬件技术动态地从指令流中移除不必要的序列化操作，从而提高并发执行效率。在*推测性锁省略*（SLE）中，硬件动态地识别同步操作，预测它们是不必要的，并将其省略。通过移除这些操作，程序的行为就好像其中不存在同步一样。当然，这样做可能会在需要同步以确保正确性的情况下破坏程序。这种情况可以通过预先存在的缓存一致性机制来检测，而*无需执行同步操作*。在这种情况下，程序会进行恢复，并显式地获取锁。只有当硬件确定序列化对于正确性是必需的时，才会执行同步操作。

安全动态锁移除是通过利用锁和临界区在常见实现方式中的特性来实现的。如果锁的获取和释放之间的内存操作看起来是原子性的，那么对应于锁的获取和释放的两次写入操作就可以被忽略，因为第二次写入（锁释放）会撤销第一次写入（锁获取）所做的更改。第 3.3 节详细讨论了这一概念。原子性违例（将在 第 5.3 节进一步讨论）可以使用大多数现代处理器中已经实现的缓存一致性协议来检测。

SLE具有以下主要特点：

1. 支持高度并发的多线程执行

   多个线程可以同时执行由同一锁保护的关键代码段。此外，正确性判定无需获取（或修改）该锁。

2. 简化了正确的多线程代码开发

   程序员可以使用保守同步来编写正确的多线程程序，而不会对性能造成显著影响。如果同步并非正确性所必需，则程序的执行行为将如同未使用同步一样。

3. 易于实施

   SLE 可以完全在微架构中实现，无需指令集支持，也无需系统级修改（例如，无需更改缓存一致性协议），并且对程序员透明。现有的同步指令会被动态识别。程序员无需学习新的编程方法，可以继续使用他们熟悉的同步例程。该技术可以集成到现代处理器设计中，且与系统和缓存一致性协议无关。

*据我们所知，这是第一个提出的技术，可以在不*执行获取和释放锁操作，*也不*要求独占锁变量的情况下，从动态执行中移除动态不必要的保守同步操作。

在第 3 节和 第 4 节中，我们讨论了 SLE 的概念，并在 第 5 节中提供了实现策略。所需的许多额外功能要么存在于现代微架构中，要么涉及为其他微架构优化而开发的成熟技术。

PDF
帮助

第二部分
背景

在本节中，我们将介绍多线程编程中锁定和性能/复杂性权衡的背景知识，这些知识可以从 SLE 中受益。

2.1 多线程编程中的性能/复杂度权衡

保守锁。在任何多线程编程工作中，编程复杂度都是需要考虑的最重要问题[7]，因此必须谨慎确保线程间的正确同步和交互。缺乏适当的同步会导致程序执行错误。为了确保正确性，程序员通常会依赖保守锁，但这往往会牺牲性能。

锁粒度。精心设计的程序必须选择合适的锁级别，以优化性能和程序正确性推理的便捷性之间的权衡。早期的并行程序通常使用较少的锁，程序员无需考虑程序的正确性，因为所有内存都受到这些锁的保护。随着并行性的提高和锁的频繁使用，性能下降，因此需要使用更细粒度的锁。更细粒度的锁可以提高性能，但会增加编程复杂性，并使程序管理变得困难。

线程不安全的遗留库。在某些环境中，不可重入的遗留软件二进制文件是造成序列化的一个重要原因。如果线程调用了一个不具备线程处理能力的库，则会使用全局锁定机制来防止冲突，从而导致访问序列化，最终造成性能下降。

2.2 微处理器中的互斥

互斥通常使用原子读-修改-写原语来实现，例如 SWAP、COM-PARE&SWAP、LOAD-LOCKED/STORE-CONDITIONAL (u.zsc) 和 EXCHANGE。这些指令允许处理器（有时是有条件地）原子地交换寄存器值和内存位置。

最简单的互斥结构是 TEST&SET 锁。TEST&SET 对内存位置执行原子交换。TEsT&TEsT&sSET [32]是 TEST&SET 的扩展，它在尝试执行 TEST&SET 操作之前，会读取锁以测试其有效性。图 2展示了 TEST&TEST&SET 序列的一个示例实现。

尽管已经提出了许多硬件和软件锁结构，但 TEST&TEST&SET 锁的简洁性和可移植性使其非常流行。硬件架构手册推荐[8]、[10]、[33]、[18]使用这些锁，数据库厂商也被建议 [22]使用这些简单的锁作为可移植的锁定机制（当然，在某些情况下也会使用其他一些软件原语）。POSIX 线程标准建议在诸如 pthread_mutex_lock() 之类的库调用中实现同步，而这些调用实现了 TEST&SET 或 TEST&TEST&SET 锁。

图 2. - 使用 alpha isa [9] 的典型锁获取和释放代码。这是一个测试&测试&设置锁结构。实例 i 到 7 构成锁获取，实例 16 构成锁释放。

图2。使用 alpha isa [9] 的典型锁获取和释放代码。这是一个测试&测试&设置锁结构。实例 i 到 7 构成锁获取，实例 16 构成锁释放。

第三部分
启用并发临界区

本节讨论推测性锁省略（Speculative Lock Elision）的概念。为了便于解释，我们使用图 2中的代码序列和 LL/SC 同步机制；这些概念同样适用于其他同步原语。我们首先讨论锁强制的虚假依赖。在 3.2 节和 3.3 节中，我们将展示如何克服这些依赖，并在 3.4 节中通过一个示例进行详细讲解。

3.1 锁如何引入虚假依赖关系

锁是一种控制变量，它决定线程是否可以执行临界区——它强制线程间存在控制依赖关系，但本身并不产生任何实际结果。此外，锁在单个线程内部也形成数据依赖关系——锁的值决定了线程的控制流。这种由锁强制执行的控制依赖关系表现为数据依赖关系，因为锁是一个由线程检查和操作的内存位置。这种依赖关系类似于数据依赖关系，例如一条指令等待其逻辑上依赖于的前置指令完成。

PDF
帮助

3.2 如何消除因锁而产生的错误依赖关系

原子性是指临界区内的所有操作看起来都是瞬间完成的。这种瞬间完成的*假象*至关重要。通过获取锁，线程可以阻止其他线程观察到临界区内的任何内存更新。虽然这种传统方法可以轻易保证临界区内所有更新的原子性，但这只是保证原子性的一种方法。

如果能够通过其他方式保证临界区内所有内存操作的原子性，则可以省略锁并并发执行临界区。为了保证原子性，临界区内必须满足以下条件：

1. 在推测执行的临界区中读取的数据，在推测执行的临界区完成之前，不会被其他线程修改。
2. 在推测执行的临界区中写入的数据，在推测执行的临界区完成之前，其他线程无法访问（读取或写入）该临界区中写入的数据。

处理器可以通过确保临界区内线程执行的部分更新不被其他线程观察到，从而在*不获取锁的情况下，为临界区内的内存操作提供原子性的假象*。这样，整个临界区看起来就像是原子性地执行的，程序语义也得以保持。

关键在于，如果硬件能够为临界区内的所有内存操作提供原子性的假象，那么并非总是需要获取锁才能正确执行。如果发生数据冲突，例如两个线程竞争同一数据（读取操作除外），则无法保证原子性，此时需要获取锁。如第 5 节所示，线程间的数据冲突可以通过现有的缓存协议实现进行检测。任何不满足上述两个条件的执行都不会被架构终止，从而保证了正确性。

从算法角度来看，序列如下：

1. 当检测到锁获取操作时，处理器*预测*临界区中的内存操作将以原子方式发生，并省略锁获取操作。
2. 推测性地执行临界区并缓冲结果。
3. 如果硬件无法提供原子性，则触发错误推测，恢复并显式获取锁。
4. 如果遇到锁释放操作，则原子性未被违反（否则之前就会触发错误推测）。省略锁释放操作，提交推测状态，并退出推测临界区。

省略锁获取操作会使锁处于释放状态，允许其他线程应用相同的算法并推测性地进入临界区。即使在获取或释放锁时锁本身未被修改，临界区的语义仍然得以保持。

在步骤 3 中，处理器可以选择在显式获取锁之前，尝试再次执行算法有限次。我们将此次数称为*重启阈值*。由于达到重启阈值后会显式获取锁，因此始终可以保证向前推进。

上述算法要求处理器识别锁获取和释放操作。如第 2.2 节所述，锁获取操作使用底层同步指令实现。这些指令并非总是仅用于实现锁获取。此外，锁释放操作使用普通的存储操作实现。因此，处理器缺乏关于操作是锁获取还是锁释放的*精确*信息，而只能观察到一系列加载、存储和底层同步原语，并只能*预测*它们是锁操作。下一节我们将展示，即使没有精确的语义信息，为什么*仍然*可以移除预测的锁获取和释放操作。

3.3 通过省略静默存储对来移除锁获取和释放

锁获取和释放操作包含存储操作。如果锁为 FREE，则锁获取*操作会将*锁标记为 HELD。锁释放*操作*会将锁标记为 FREE。图 3以三列形式显示了 SLE 下的内存引用。指令按程序顺序编号。第一列显示程序员的视图，第二列显示处理器执行的操作，第三列显示位置 _ 的值。锁_ 从不同的帖子中可以看出。

图 3. - 静默存储对省略。如果 i16 将 _lock_ 的值恢复到 i6 之前的值（即 B 返回的值），并且 i8 到 i15 相对于其他线程似乎是原子执行的，则实例 i6 和 i16 可以被省略。尽管推测线程省略了 i6，但它仍然观察到 HELD 值（因为单个线程内的程序顺序要求），而其他线程观察到的是 FREE 值。

图 3。静默存储对省略。如果 i16 恢复了 _ 的值，则可以省略实例 i6 和 i16。锁_ 的值在 i6 之前（即 B 回的值），而 i8 到 i15 相对于其他线程似乎是原子执行的。尽管推测线程省略了 i6，但它仍然观察到 HELD 值（因为单个线程内的程序顺序要求），而其他线程观察到的是 FREE 值。

如果 i3 返回 FREE，则 i6 将 HELD 写入位置 _锁_. i16 通过将其标记为 FREE 来释放锁。锁释放 (i16) 后，_ 的值锁_ 与获取锁开始时（即 i6 之前）的值相同 - i16 恢复 _ 的值锁_ 恢复到 i6 之前的值。我们利用同步操作的这一特性来省略锁的获取和释放。如果临界区内存操作是原子性的，那么存储 i6 和 i16 就构成一对*静默操*

PDF
帮助

作。i6 执行的架构更改会被 i16 撤销。当它们作为一对执行时，存储操作是静默的；单独执行时则不是。位置 _锁_ 不能被其他线程修改，否则 i6 和 i16 无法形成静默配对。注意：其他线程可以读取内存位置 _。锁_。

上述观察表明，SLE 算法无需依赖程序语义信息，特别是操作是获取锁还是释放锁。只需观察加载和存储序列以及读取和要写入的值，即可完成锁省略。如果任何指令序列与图 3中第 2 列和第 3 列的模式匹配，则位置 _锁如果 _ 未被其他线程修改，且临界区内的内存操作似乎是原子执行的，则对应于 i6 和 i16 的两次存储操作将被省略。位置 _锁_ 永远不会被修改，其他线程可以继续执行，而无需对 _ 的值进行序列化。锁_。

因此，在第 3.2 节的算法中增加了一个额外的预测。对于预测为锁获取的存储操作，处理器预测该存储操作所做的更改很快就会被另一个存储操作撤销，并且没有其他线程会修改相关位置。如果情况确实如此，并且由于整个序列被全局观察到是原子性的，则可以省略这两个存储操作。我们使用过滤器来确定候选的加载/存储对。例如，在我们的实现中，仅考虑指令 Idl_l 和 st l_c（它们通常成对出现）。stl_c 存储操作被推测性地省略，并识别出符合 图 3模式的未来存储操作。

SLE 的完整算法如下：

1. 如果对某个地址进行候选加载（ldl_l）后，紧接着对同一地址进行存储（获取锁的 stl_c），则预测很快就会发生另一次存储（释放锁），将内存位置的值恢复到此存储（获取锁的 stl_c）之前的值。

2. 预测关键区域中的内存操作将以原子方式执行，并省略锁的获取。

3. 推测性地执行临界区并缓冲结果。

4. 如果硬件无法提供原子性，则触发错误推测，恢复并显式获取锁。

5. 如果在步骤 1 中看到了第二次存储（释放锁），则原子性未被违反（否则会更早触发误推测）。省略释放锁的存储、提交状态和退出推测临界区。

注意，在上述修改后的算法中，硬件无需了解内存访问是否针对锁变量的语义信息。硬件仅跟踪值的变化并观察来自其他线程的请求。如果步骤 5 的存储操作与步骤 1 中概述的值要求不符，则直接执行存储操作。如果存储操作完成后原子性仍然得到保证，则可以安全地退出临界区。

3.4 推测性锁省略算法示例

图 4展示了 SLE 在我们之前 图 1a示例中的应用。修改后的控制流显示在右侧，其中省略了指令 6 和 16。所有线程均不进行序列化。指令 1 和 3 引入了 _锁_ 以共享状态存入缓存。指令 6 被省略，修改后的控制流被推测性地执行。位置 _锁_ 会监控其他线程的写入操作。处理器执行的所有加载操作都会被记录。所有执行的存储操作都会被临时缓冲。如果执行到第 16 条指令时没有违反原子性，则 SLE 成功。

如果线程无法记录两个存储之间的访问，或者硬件无法提供原子性，则会触发错误推测，并从指令 6 重新开始执行。重新开始时，如果已达到重新开始阈值，则执行将以非推测的方式进行，并获取锁。

第四部分
为什么 SLE 能正常工作?

现在我们来讨论为什么 SLE 即使在软件缺乏精确信息的情况下，也能保证正确执行，并且与嵌套层级和内存顺序无关。如前所述，SLE 包含两个预测：

1. 在执行一次存储操作时，预测接下来会执行另一次存储操作，并撤销本次存储操作所做的更改。该预测无需实际执行存储操作即可完成，但需要监控存储操作的内存位置。如果预测正确，则省略这两次存储操作。

2. 预测在两个省略的存储操作所界定的窗口内的所有内存操作都是原子性的。该预测通过使用第 5.3 节中述的缓存一致性机制，检查第 3.2 节中概述的条件来实现。

上述预测并不依赖于程序的语义（锁预测器用于识别加载/存储操作作为预测 1 的候选对象，但这并非该思路的必要组成部分，软件也可以提供这些提示）。此外，任何部分更新都不会对其他线程可见。这样做保证了临界区语义。存储省略之所以有效，是因为架构状态保持不变。无论是否使用存储省略，第二次省略存储操作结束时的架构状态都相同。

PDF
帮助

图 4. - 推测性锁省略算法示例。通常情况下，会执行分支 11，从而跳过存储指令 12。右侧图中灰色部分不会被执行。指令 6 和 16 被省略，代码序列执行时 i1 和 i8 之间没有执行任何分支。

图 4。推测性锁省略算法示例。通常情况下，会执行分支 11，从而跳过存储指令 12。右侧图中灰色部分不会被执行。指令 6 和 16 被省略，代码序列执行时 i1 到 i8 之间没有执行任何分支。

如果另一个线程通过写入操作显式地获取了锁，则会触发错误推测，因为所有推测线程都会自动观察到该写入操作。这可以简单地保证即使一个线程正在进行推测而另一个线程获取了锁，结果仍然正确。

嵌套锁。虽然可以将省略算法应用于多个嵌套锁，但我们只将其应用于其中一层（可以是任何一层，不一定是最外层），并且该层内的任何锁操作都被视为推测性内存操作。

内存一致性。由于SLE下的推测性内存操作具有原子性，因此不存在内存排序问题。无论采用何种内存一致性模型，线程将一组*原子*内存操作插入到全局内存操作顺序中始终是正确的。

第五节
实施SLE

我们已经展示了如何使用 SLE 动态地移除不必要的同步操作。现在，我们将展示如何使用成熟且常用的技术来实现 SLE。

SLE 类似于分支预测和其他推测执行技术。省略的锁获取可以看作是一个预测的分支，而省略的锁释放则类似于分支的解析。然而，SLE 并不要求处理器支持乱序执行，而仅仅要求处理器具备*推测性地放弃*指令的能力。换句话说，它不需要维护指令间的依赖关系信息。

SLE 的实现包括四个方面：1）启动推测，2）缓冲推测状态，3）错误推测条件及其检测，以及 4）提交推测内存状态。

5.1 发起投机

使用过滤器来检测推测的候选对象（例如，Idl_l/stl_c这些锁对）由程序计数器索引。此外，每个锁对都被分配一个置信度指标。如果处理器预测锁将被持有，则它假定由于自身无法省略锁，该锁必须由另一个处理器获取。在这种情况下，处理器不会启动推测。这是一种保守的方法，但有助于防止在极端情况下性能下降。更精确的置信度估计是未来研究的一个重要方向。

5.2 缓冲推测状态

为了从 SLE 误判中恢复，必须缓冲寄存器和内存状态，直到 SLE 得到验证。

推测性寄存器状态
处理寄存器状态的两种简单方法是：

1. *重排序缓冲区（ROB）：*使用重排序缓冲区[35]的优势在于可以利用已用于分支预测错误的恢复机制。然而，ROB 的大小限制了临界区的大小（以动态指令的数量计）。

2. *寄存器检查点：*这可以是依赖关系映射（物理寄存器的释放方式可能存在某些限制），也可以是架构寄存器状态本身。如果发生错误推测，则会恢复检查点。使用检查点消除了临界区大小的限制：指令可以安全地更新寄存器文件、推测性地退出，并从 ROB 中移除，因为存在正确的架构寄存器检查点，以便在发生错误推测时进行恢复。重要的是，只需要一个这样的检查点，它在 SLE 序列开始时执行。

推测性记忆状态
尽管大多数现代处理器支持推测性加载执行，但它们并不支持推测性地撤销存储（即，推测性地写入内存系统）。为了支持推测性加载执行，我们扩展了现有的处理器写缓冲区（位于处理器和 L1 缓存之间），以缓冲推测性内存更新。在锁省略验证通过之前，推测性数据不会从写缓冲区提交到下层内存层次结构。如果推测错误，则写缓冲区中的推测性条目将被失效。

PDF
帮助

此外，在SLE架构下，推测性写入现在可以合并到写缓冲区中，而*无需考虑*内存一致性模型。这是因为，对于成功的推测，所有内存访问都保证以*原子方式*完成。只有写缓冲区的大小限制了临界区中修改的唯一缓存行的数量，而没有限制临界区中执行的存储指令的动态数量。

5.3 误投条件及其检测

导致错误推测的两个原因是：1）违反原子性；2）由于资源有限而导致的违反原子性。

原子性违背

原子性违例（3.2 节）可以使用现有的缓存一致性方案进行检测。缓存一致性是一种将内存更新传播到其他缓存并使内存操作对其他缓存可见的机制。基于失效的一致性协议保证在执行存储操作时，本地缓存中存在该内存块的独占副本。大多数现代处理器已经在其本地缓存层次结构中实现了某种形式的基于失效的一致性。因此，检测不同处理器之间内存操作冲突的基本机制已经存在。现在需要一种机制来记录临界区内读取和写入的内存地址。

在某些处理器中（例如 MIPS R10K 和 Intel Pentium 4），加载/存储队列 (LSQ) 会监听缓存接收到的任何外部失效操作，以支持更严格的内存一致性实现[12]。如果 SLE 使用 ROB 方法，则无需额外的机制来跟踪推测读取的内存位置的外部写入操作——LSQ 已被监听。

如果采用寄存器检查点方法，仅靠 LSQ 无法检测 SLE 期间的加载违例，因为加载操作可能会推测性地退出并离开 ROB。在这种情况下，可以为每个缓存块添加一个*访问位*。SLE 期间执行的每次内存访问都会标记相应块的访问位。收到外部请求时，会并行检查该位和缓存标签。任何对已设置访问位的块的外部失效操作，或者对已设置访问位的排他状态块的外部请求，都会触发推测错误。该位可以与标签一起存储，并且由于为了保持一致性，已经执行了标签查找来侦听缓存，因此位比较不会产生额外开销。

该方案与缓存级别数量无关，因为所有缓存都保持一致性，并且任何更新都会通过现有协议自动传播到所有一致的缓存。

对于误判和提交操作，缓存中所有块的访问位都会被取消设置；这可以通过诸如闪存失效[26]之类的技术来实现。对于激进的乱序处理器，当候选存储（用于锁获取省略）被解码时，处理器发出的任何后续加载操作都会标记相应的访问位。该加载操作实际上可能并不属于临界区，但保守地标记它始终是正确的。管道中可以允许多个候选存储（预测的锁获取），只要核心内存中存在候选存储，加载操作就会保守地标记访问位。

资源限制导致的违规行为

资源有限可能导致误判，例如没有足够的缓冲区空间来存储推测性更新，或者无法监控已访问的数据以提供原子性保证。以下四种情况会导致误判：

1. 缓存容量有限。如果使用寄存器检查点，缓存可能无法跟踪所有内存访问。

2. 写入缓冲区容量有限。已修改的唯一缓存行数量超过了写入缓冲区容量。

3. ROB 大小有限。如果采用检查点方法，ROB 大小就不是问题。

4. 未缓存的访问或事件（例如，某些系统调用），处理器无法跟踪请求。

对于条件 1、2 和 3，并非总是需要重启。处理器可以简单地写入它所获取的锁标记。写入完成后，如果原子性得到保证，则可以提交推测，处理器无需重启即可继续执行。

5.4 提交推测性记忆状态

我们已经讨论了如何恢复和提交架构寄存器状态、如何在写缓冲区中缓冲推测性存储状态，以及如何使用现有的缓存一致性协议检测错误推测情况。提交内存状态需要确保推测性缓冲的写入操作能够*立即*提交并对内存系统可见（以保证原子性）。

缓存包含两个方面：1）状态，2）数据。缓存一致性协议决定了缓存块状态的转换。重要的是，只要数据本身不发生推测性更改，这些状态转换就可以进行推测性转换。现代处理器正是基于此原理，执行推测性加载和其他性预取（将排他状态的数据加载到缓存中的操作）。我们在执行原子内存提交时，正是利用了缓存的这两个方面，*而无需*对缓存一致性协议进行任何更改。

当向写缓冲区添加推测性存储时，会向内存系统发送一个独占请求。该请求会触发一致性协议中预先存在的转换，并将缓存块以独占状态移入本地缓存。请注意，*缓存块数据并非推测性的——推测性*数据会被缓冲在写缓冲区中。当临界区结束时，写缓冲区中所有推测性条目在缓存中都会有对应的独占状态块，否则会更早触发误推测。此时，写缓冲区被标记为具有最新的架构状态。

PDF
帮助

写入缓冲区需要具备从其他线程获取数据以响应请求的功能。这并非关键路径上的操作，写入缓冲区的数据可以根据需要延迟加载到缓存中。之所以能够实现即时提交，是因为将写入缓冲区标记为最新状态的过程涉及为所有推测性更新和缓存的数据块设置一位独占权限。

图 5显示了两个设计要点：（a）使用 ROB 存储推测状态，（b）使用额外的寄存器检查点和带有缓存标记的访问位。

第六节
评估方法

多线程基准测试的控制流会因底层一致性机制的不同而有所差异，性能提升也高度依赖于底层协议的实现。为了解决这个问题，我们评估了多种配置。表 1显示了三种多处理器系统的参数：a) 芯片多处理器 (CMP)；b) 更传统的总线系统 (SMP)；c) 目录系统 (DSM)。总线协议基于 Sun Gigaplane [6]，目录协议基于 SGI Origin 2000 [25]。处理器采用全存储排序 (TSO) 作为内存一致性模型。已执行的存储操作会被写入写缓冲区，并按程序顺序在架构上可见。在一致性事件发生时，所有正在进行的加载操作都会被监听，并在必要时重放。我们使用单个寄存器检查点进行 SLE 寄存器恢复，并使用由程序计数器索引的 32 项锁预测器。

图 5 - 推测性锁省略的两种设计方案。阴影框代表处理器。修改部分和附加数据路径以斜体灰色线条显示。

图 5。推测性锁省略的两个设计要点。阴影框代表处理器。修改部分和附加数据路径以斜体灰色线条显示。

6.1 仿真环境

我们使用 SimpleMP，这是一款用于运行多线程二进制文件的执行驱动型模拟器。该模拟器源自 Simplescalar 工具集[4]。我们重写了该模拟器，使其能够精确地模拟多处理器配置中的乱序处理器和详细的内存层次结构。为了精确地模拟一致性和内存一致性事件，处理器对缓存和写缓冲区中的数据进行操作（读写）。内存系统中模拟了争用。为了确保模拟的正确性，功能检查模拟器在详细的时序模拟器之后运行，*仅*用于检查正确性。功能检查模拟器在其独立的内存和寄存器空间中工作，可以验证 TSO 实现。

6.2 基准测试

我们使用一个简单的微基准测试和六个应用程序（表 2）来评估我们的方案。该微基准测试包含 N 个线程，每个线程递增一个唯一的计数器 (2^{16})/N 次，所有 N 个计数器都由单个锁保护。这是传统锁的最坏情况，但清楚地展示了我们方案的潜力。对于这六个应用程序，我们使用mp3d，巴恩斯和乔列斯基来自 SPLASH [34]套件和辐射度，水-nsq， 和海洋来自 SPLASH2 [40]套件。

表 1.模拟机器参数。

表 1.- 模拟机器参数。

表2：基准测试

表2：基准测试

选择这些应用程序是因为它们具有不同的锁行为、内存访问模式和临界区行为。这些基准测试已进行适当的填充，以减少伪共享。为了研究 SLE 对锁密集型基准测试的影响，我们使用了 Mp3 d 的锁定版本[20]。该版本的 Mp3 d 会频繁地同步到几乎没有竞争的锁，因此即使使用大型重排序缓冲区，也无法掩盖锁访问延迟。斯基和辐射度有经常被访问的工作队列。海洋具有条件更新代码序列。Barnes 算法存在较高的锁和数据争题，而水-nsq几乎没有争议。

这些基准测试用例都经过优化，便于共享，因此在大多数情况下通信量很少。我们感兴趣的是，即使对于这些经过精心调优的基准测试用例，我们的方案是否仍然具有鲁棒性和潜力。

PDF
帮助

第七节
结果

对于频繁执行同步操作的应用而言，SLE 可以带来显著的性能提升。我们首先在微基准测试中讨论 SLE 的性能，然后分析其在一系列应用中的性能。

7.1 微基准测试结果

图 6以 y 轴表示微基准测试的执行时间（CMP 配置），x 轴表示处理器数量的变化。正如预期，由于严重的竞争，传统锁机制下的性能迅速下降。即使计数器更新之间不存在冲突，乱序处理器也无法利用这一特性，因为锁获取序列会掩盖这种并行性，并且性能受限于锁获取操作。然而，采用 SLE 机制后，硬件会自动检测到不需要锁并将其省略。由于 SLE 不需要通过写入锁来验证推测，因此实现了完美的扩展性。

图 6. - CMP 的微基准测试结果。

图 **6**。CMP的微基准测试结果。

7.2 基准测试结果

SLE 模式下一个可调整的参数是重启阈值。该参数决定了处理器在显式获取锁以执行临界区之前可以经历多少次误推测。我们针对不同的阈值进行了实验，并展示了阈值为 1 时的结果——处理器在 SLE 模式下发生原子性违规后会重启一次，并再次尝试获取锁。需要注意的是，如果某个处理器持有锁，其他处理器不会尝试 SLE，而是会根据程序的原始控制流顺序，在该锁上进行自旋（或等待）（等待空闲锁后再尝试 SLE）。这样做可以确保持有锁的处理器不会受到其他处理器对临界区数据的访问干扰。

滑锁

图 7显示了重启阈值为 1 时动态锁获取/释放对被省略的百分比。大部分动态锁获取操作都被省略了。这种省略并不总是能转化为更好的性能，因为这些操作可能并非程序的关键路径，但它证明了我们技术的有效性。阈值为 0（首次错误推测即重启）时，被省略的锁获取操作减少了 10% 到 30%。在 Barnes 算法中，受保护数据的争用很高，重复重启往往会导致冲突。因此，对于 Barnes 算法，被省略的锁数量较少。

图 7. - 动态锁定获取/释放的百分比（省略）

图 **7**。动态锁定获取/释放的百分比已省略

表现

图 8显示了 8 个和 16 个处理器的归一化执行时间。y 轴表示归一化执行时间（启用 SLE 的并行执行周期数/禁用 SLE 的并行执行周期数）。I*下方的数字表示加速比*。对于每个柱状图，上半部分表示锁变量访问所占的贡献（LOCK-PORTION），下半部分表示其余部分（NON-LOCK-PORTION）。柱状图上方的小数部分表示启用 SLE 时的归一化执行时间。

对于某些配置，优化方案的非锁定部分比基准方案的相应非锁定部分更大。这是因为有时移除锁会将其他内存操作置于关键路径上。之前与获取锁操作重叠的关键段内的数据的推测性加载现在暴露出来，导致处理器停顿。

观察到的性能提升主要有三个原因：1）并发临界区执行，2）观察到的内存延迟减少，以及3）内存流量减少。

并行临界区执行

在海洋含量，辐射度， 和乔列斯基即使存在锁竞争，临界区有时也能在不获取锁的情况下正确执行，线程执行也不会因锁的获取而串行化。内存延迟越大，这种优势就越明显，因为由获取锁延迟引起的串行化延迟也越大。

PDF
帮助

图 8. - CMp、SMp 和 DSM 配置下 8 线程和 16 线程的归一化执行时间。y 轴表示归一化并行执行时间。每个基准测试均显示三对柱状图，分别对应 CMP、SMP 和 DSM 配置。每对柱状图的第一条柱状图对应基准情况，第二条柱状图对应 SLE 配置。对于每条柱状图，上半部分对应锁变量访问所占的时间（LOCK-PORTION），下半部分对应其余部分（NON-LOCK-PORTION）。柱状图上方的分数表示 SLE 配置下的归一化执行时间。LOCK-PORTION 部分还包含等待锁的时间。所有归一化均以基准情况（每对柱状图的左侧柱状图）为基准。

图 8。图示为 CMp、SMp 和 DSM 配置下 8 线程和 16 线程的归一化执行时间。y 轴表示归一化并行执行时间。每个基准测试均显示三对柱状图，分别对应 CMP、SMP 和 DSM 配置。每对柱状图的第一条柱状图对应基准情况，第二条柱状图对应 SLE 情况。对于每条柱状图，上半部分表示锁变量访问所占的时间（LOCK-PORTION），下半部分表示其余部分（NON-LOCK-PORTION）。柱状图上方的分数表示 SLE 情况下的归一化执行时间。LOCK-PORTION 还包含等待锁的时间。所有归一化均以基准情况（每对柱状图的左侧柱状图）为基准。

观察到的记忆延迟减少
通常，获取锁会导致缓存未命中，并向内存系统发出写入请求，而这些延迟无法与临界区内的未命中完全重叠。SLE 允许锁在本地缓存中共享，因此处理器不会察觉到获取锁的未命中。几乎所有基准测试都受益于此。Water-nsq 的收益不大，因为在锁省略之后，重叠的临界区未命中会被暴露出来。

减少内存流量
如果一个锁被获取并以独占状态保存在处理器的缓存中，那么来自另一个处理器的锁请求将需要总线/网络消息（一条用于读取锁，另一条用于设置锁）。因此，对于频繁同步的基准测试，减少锁请求可以降低内存流量，从而带来性能提升。通过不获取锁，锁在各个处理器上以共享状态保存，从而消除了锁未命中流量。mp3 d 基准测试从中获益最大，因为它避免了频繁的同步。由于某些锁确实会经历无法重叠的缓存未命中延迟，因此仍然存在一些锁访问延迟。SMP 和 DSM 版本比 CMP 版本获益更多，因为它们的大缓存可以容纳工作集，从而减少了锁的读取未命中（以及内存流量）。对于 CMP 版本，缺少大缓存会造成性能损失，因此由于 Li 缓存会遭受冲突和容量未命中，导致更多处于干净状态的锁被驱逐。
误投效应
在我们的实验中，由于容量和缓存冲突（由于关联度有限）导致的误推测发生率低于0.5%。对于原子性冲突，我们会提前重启并获取锁，从而降低误推测的惩罚。非锁定部分也包含一些由误推测引起的内存延迟。

对重启阈值的依赖性
重启阈值设为 0 时，加速比设为 1 时最多低 25%。提高阈值可以显著提升某些基准测试的性能。然而，对于拥有 16 个处理器的 Barnes 系统，阈值设为 5 时，性能反而下降了 10%。这是因为处理器错误推测会引入一致性协议干扰，从而增加临界区内数据的延迟。选择较低的阈值（0 或 1）的优势在于可以最大限度地减少因重复错误推测而导致的性能下降。对于这些基准测试，当阈值设为 1 时，性能下降的情况很少见（损失 1%）。即使省略了更多锁，提高阈值有时也可能导致性能略微下降，这是因为临界区内的数据访问干扰。动态调用中用于选择重启阈值的预测器是未来的研究方向。

第 8 节
相关工作

Lamport 引入了无锁同步[24]，并给出了允许多个线程在不加锁的情况下操作同一数据结构的算法。对无锁数据结构的操作支持并发更新，且不需要互斥。无锁数据结构已被广泛研究 [3]、 [15]。实验研究表明，无锁数据结构的软件实现性能不如基于锁的实现，这主要是因为在必要时回滚需要进行大量的数据复制 [1]、 [16]。事务内存 [16]和 Oklahoma 更新协议 [37]是最初提出的为实现无锁数据结构提供硬件支持的方案。两者都为程序员提供了访问这些数据结构的特殊内存指令。尽管这些方案在概念上很强大，但它们需要指令集支持和程序员的参与。程序员必须学习正确使用新指令，并且这些方案需要对一致性协议进行扩展。此外，现有的程序二进制文件无法从中受益。这些方案依赖于软件支持来保证进展。这些提议都是对 Jensen 等人 [19]最初提出的 LOAD-LINKED 和 STORE-CONDITIONAL 指令的直接推广。LOAD-LINKED/STORE-CONDITIONAL 组合允许对一个字进行原子读-修改-写序列。

与上述方案不同，我们的方案无需修改指令集、扩展一致性协议或提供程序员支持。因此，在大多数情况下，当竞争的临界区执行之间没有冲突时，我们可以以无锁方式运行未经修改的二进制文件。我们无需为向前进提供特殊支持，因为对于冲突，我们只需回退到原始代码序列，并以常规方式获取和释放锁即可。

数据库并发控制领域已开展了大量研究，Thomasian [39]对此进行了很好的总结并提供了更多参考文献。乐观并发控制 (OCC) 由 Kung 和 Robinson [23]提出，作为数据库管理系统中锁机制的替代方案。OCC 包含一个读取阶段，在此阶段访问对象（并可能更新这些对象的私有副本），随后是一个串行化的验证阶段，用于检查数

PDF
帮助

据冲突（与其他事务的读/写冲突）。如果验证成功，则进入写入阶段。尽管研究广泛，OCC 作为一种并发控制机制在数据库系统中并不常见。Mohan [28]对 OCC 方法所涉及的问题及其缺点进行了精彩的讨论，这些缺点使其不适用于高性能数据库系统。数据库系统 [29]的特殊要求和保证使得 OCC 难以用于高性能应用。为了提供这些保证，必须在软件中存储大量的状态信息，从而导致巨大的开销。此外，OCC 的验证阶段是串行化的。

我们的方案与数据库 OCC 方案截然不同。我们并非提供基于锁的同步方案的替代方案：我们检测动态实例，并在这些同步操作不必要时将其*移除*。对临界区的要求远低于数据库系统。由于我们不需要显式获取锁来判断成功，因此我们没有串行化的验证阶段。

已有微架构支持推测性退役[31]、 [13]以及在缓存中缓冲推测性数据 [11]、 [14]、 [36]的研究。我们的工作可以利用这些技术并与之共存。然而，这些早期技术均无法动态地从动态指令流中移除保守同步。预测锁的获取和释放此前已有研究提出，我们也采用了类似的技术 [30]。

我们提出的静默对省略方案是对 Lepak 和 Lipasti [27]提出的*静默存储*方案的扩展。他们省略了单个静默存储操作，而我们省略了那些单独执行时并非静默，但成对执行时却是静默的存储对。

第九节
总结发言

我们提出了一种微架构技术，用于消除动态指令流中不必要的串行化。其核心思想在于，无需获取锁，只需观察锁即可。利用我们的技术，锁操作所隐含的控制依赖性被转换为各个并发临界区之间的真实数据依赖性。因此，由程序员主导的静态分析所掩盖的动态不必要且保守的锁机制，通过基于硬件的动态分析得以展现。

所提出的技术无需对一致性协议进行任何更改。此外，也无需程序员或编译器支持，更无需更改指令集。内存操作原子性这一关键概念使得该技术能够集成到任何处理器中，而无需考虑内存一致性，因为其正确性无需依赖于内存顺序即可得到保证。

我们认为我们的方案是实现高性能多线程编程的重要一步。随着多进程的日益普及，有必要为程序员提供支持，以便他们能够利用这些多进程特性来提升功能和性能。推测性锁消除（Speculative Lock Elision，简称 SLE）允许程序员使用频繁且保守的同步机制来轻松编写*正确的*多线程代码；我们的技术能够自动且动态地移除不必要的同步实例。同步仅在确保代码正确性所必需的情况下执行；并且这种同步机制的存在不会降低性能。由于SLE是一种纯粹的微架构技术，因此它可以集成到任何系统中，而无需依赖一致性协议或考虑系统设计问题。

致谢

我们衷心感谢 Mark Hill、Mikko Lipasti 和 David Wood 就本文观点进行的宝贵讨论。我们感谢 Brian Fields、Adam Butts、Trey Cain、Timothy Heil、Mark Hill、Herbert Hum、Milo Martin、Paramjit Oberoi、Manoj Plakal、Eric Rotenberg、Dan Sorin、Vijayaraghavan Soundararajan、David Wood 和 Craig Zilles 对本文草稿提出的宝贵意见。Jeffrey Naughton 和 C. Mohan 为我们提供了有关数据库系统中 OCC 的信息。

作者
数据
参考
引用
关键词
指标
PDF
帮助

IEEE个人账户
更改用户名/密码

购买详情
付款方式
查看已购买的文件

个人资料
沟通偏好
职业与教育
技术兴趣

需要帮助吗?
美国和加拿大：+1 800 678 4333
全球：+1 732 981 0060
联系与支持

跟随

关于 IEEE *Xplore* |联系我们|帮助|无障碍访问|使用条款|反歧视政策| IEEE 道德报告 |网站地图| IEEE 隐私政策
IEEE 是一个公共慈善机构，也是世界上最大的技术专业组织，致力于推进技术发展，造福人类。

© 2026 年 IEEE 版权所有 – 保留所有权利，包括文本和数据挖掘以及人工智能和类似技术的训练的权利。

PDF
帮助